

Exam templates: copy the model, then change only the marked parameters/lines. All snippets are designed to be mixed into pipelines.

1. UNIVERSAL STARTERS

IMAGE LOAD + CHECK

```
import cv2, numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("image.jpg")
if img is None: print("Image not found")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

SHOW ONE / MANY

```
plt.imshow(rgb); plt.axis("off"); plt.show()
plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(gray,cmap="gray");
    plt.axis("off")
plt.subplot(1,3,2); plt.imshow(img[:, :, ::-1]);
    plt.axis("off")
plt.subplot(1,3,3); plt.imshow(cv2.cvtColor(result,
    cv2.COLOR_BGR2RGB));
plt.axis("off")
plt.show()
```

SAVE / RESIZE / CROP

```
cv2.imwrite("output.jpg", result)
small = cv2.resize(img, (500,400))
crop = img[y1:y2, x1:x2]
# dimensions
h,w = img.shape[:2]
```

2. GRAYSCALE / HISTOGRAM / ENHANCEMENT

HISTOGRAM

```
hist = cv2.calcHist([gray],[0],None,[256],[0,256])
plt.plot(hist); plt.xlim([0,256]); plt.show()
```

HISTOGRAM EQUALIZATION

```
equalized = cv2.equalizeHist(gray)
```

LOG TRANSFORM

```
g = gray.astype(np.float32)
c = 255/np.log(1+np.max(g))
log_img = c*np.log(1+g)
log_img = np.uint8(log_img)
# gamma <1 brightens; gamma >1 darkens
gamma=0.5
gamma_img=np.uint8(255*((gray/255)**gamma))
```

CONTRAST / BRIGHTNESS

```
bright = cv2.convertScaleAbs(img, alpha=1.2, beta=30)
# STATS
mean=np.mean(gray); std=np.std(gray)
mn=np.min(gray); mx=np.max(gray)
```

3. THRESHOLDING / BINARY / ROI MASK

GLOBAL

```
_, binary=cv2.threshold(gray,127,255,
    cv2.THRESH_BINARY)
# OTSU = automatic threshold
_, otsu=cv2.threshold(gray,0,255,
    cv2.THRESH_BINARY+cv2.THRESH_OTSU)
```

ADAPTIVE = uneven lighting

```
adaptive=cv2.adaptiveThreshold(
    gray,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,11,2)
# invert if object/background is reversed:
cv2.THRESH_BINARY_INV
```

RECTANGULAR ROI

```
roi=gray[y1:y2,x1:x2]
```

POLYGON ROI

```
mask=np.zeros(gray.shape,np.uint8)
pts=np.array([[100,100],[500,100],[500,400],[100,400]])
```

```
], np.int32)
cv2.fillPoly(mask, [pts], 255)
masked=cv2.bitwise_and(gray, gray, mask=mask)
```

4. BLUR / EDGES / MORPHOLOGY

```
# BLUR
gauss=cv2.GaussianBlur(gray, (5, 5), 0)
median=cv2.medianBlur(gray, 5) # salt/pepper

# CANNY
edges=cv2.Canny(gauss, 50, 150)

# KERNEL + OPENING/CLOSING
kernel=np.ones((5, 5), np.uint8)
opening=cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closing=cv2.morphologyEx(binary, cv2.MORPH_CLOSE,
    kernel)
# opening: remove small noise
# closing: fill small gaps/holes

# ERODE / DILATE
eroded=cv2.erode(binary, kernel, iterations=1)
dilated=cv2.dilate(binary, kernel, iterations=1)
# common pipeline:
gray -> blur -> threshold -> morphology -> edges
```

5. CONTOURS / OBJECT COUNT / BOUNDING BOX

```
# FIND CONTOURS
contours, _=cv2.findContours(
    binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
    area=cv2.contourArea(c)
    if area>500:
        x, y, w, h=cv2.boundingRect(c)
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)

# COUNT OBJECTS
count=0
```

```
for c in contours:
    if cv2.contourArea(c)>500:
        count+=1
    print("Objects:", count)
# perimeter
peri=cv2.arcLength(c, True)
```

CENTROID

```
M=cv2.moments(c)
if M["m00"]!=0:
    cx=int(M["m10"]/M["m00"])
    cy=int(M["m01"]/M["m00"])
    cv2.circle(img, (cx, cy), 5, (0, 0, 255), -1)
```

6. K-MEANS / COLOR SEGMENTATION

K-MEANS

```
rgb=cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
pixels=np.float32(rgb.reshape((-1, 3)))
criteria=(cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX
    _ITER, 100, 0.2)
K=3
_, labels, centers=cv2.kmeans(
    pixels, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)
centers=np.uint8(centers)
seg=centers[labels.flatten()].reshape(rgb.shape)
```

HSV RANGE MASK

```
hsv=cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower=np.array([H1, S1, V1])
upper=np.array([H2, S2, V2])
mask=cv2.inRange(hsv, lower, upper)
result=cv2.bitwise_and(img, img, mask=mask)
```

COLOR SEGMENT + CLEAN

```
kernel=np.ones((5, 5), np.uint8)
mask=cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask=cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

7. HOUGH LINES / LANES / CIRCLES**# HOUGH LINES**

```
edges=cv2.Canny(gray,50,150)
lines=cv2.HoughLinesP(
edges,1,np.pi/180,threshold=50,
minLineLength=50,maxLineGap=20)
for line in lines:
x1,y1,x2,y2=line[0]
cv2.line(img,(x1,y1),(x2,y2),(0,255,0),2)
```

LANE MODEL

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
blur=cv2.GaussianBlur(gray,(5,5),0)
edges=cv2.Canny(blur,50,150)
roi=cv2.bitwise_and(edges,edges,mask=mask)
lines=cv2.HoughLinesP(roi,1,np.pi/180,50,
minLineLength=50,maxLineGap=20)
```

HOUGH CIRCLES

```
circles=cv2.HoughCircles(
gray,cv2.HOUGH_GRADIENT,dp=1,minDist=30,
param1=100,param2=30,minRadius=10,maxRadius=100)
if circles is not None:
circles=np.uint16(np.around(circles))
for x,y,r in circles[0]:
cv2.circle(img,(x,y),r,(0,255,0),2)
```

8. AFFINE / ROTATION / TRANSLATION**# ROTATE**

```
h,w=img.shape[:2]
M=cv2.getRotationMatrix2D((w//2,h//2),30,1.0)
rot=cv2.warpAffine(img,M,(w,h))
```

ROTATE + TRANSLATE

```
M=cv2.getRotationMatrix2D((w//2,h//2),30,1.2)
M[0,2]+=50
M[1,2]+=30
out=cv2.warpAffine(img,M,(w,h))
```

CUSTOM AFFINE: 3 POINTS

```
src=np.float32([[100,100],[400,100],[100,400]])
dst=np.float32([[150,120],[450,100],[120,450]])
M=cv2.getAffineTransform(src,dst)
out=cv2.warpAffine(img,M,(w,h))
# 3 corresponding points -> AFFINE
```

9. HOMOGRAPHY / PERSPECTIVE / PANORAMA**# HOMOGRAPHY: 4+ POINTS**

```
src=np.float32([[x1,y1],[x2,y2],[x3,y3],[x4,y4]])
dst=np.float32([[a1,b1],[a2,b2],[a3,b3],[a4,b4]])
H,mask=cv2.findHomography(src,dst,cv2.RANSAC,5.0)
warped=cv2.warpPerspective(img,H,(w,h))
# 4 corresponding points -> HOMOGRAPHY
```

PERSPECTIVE TRANSFORM

```
corners=np.float32([[0,0],[w,0],[w,h],[0,h]]
).reshape(-1,1,2)
new=cv2.perspectiveTransform(corners,H)
cv2.polylines(out,[np.int32(new)],True,(0,255,0),3)
```

PANORAMA CORE**# image1 + image2:**

```
SIFT -> BF/FLANN matches -> ratio test
-> 4+ good matches -> findHomography(RANSAC)
-> warpPerspective -> place/stitch images
```

10. SIFT / MATCHING / OBJECT RECOGNITION

```
sift=cv2.SIFT_create()
kp1,des1=sift.detectAndCompute(gray1,None)
kp2,des2=sift.detectAndCompute(gray2,None)
bf=cv2.BFMatcher()
matches=bf.knnMatch(des1,des2,k=2)
good=[]
for m,n in matches:
if m.distance < 0.75*n.distance:
good.append(m)
```

DRAW MATCHES

```
match_img=cv2.drawMatches(
img1,kp1,img2,kp2,good,None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
# need >=4 good matches for homography
```

OBJECT RECOGNITION

```
src=np.float32([kp1[m.queryIdx].pt for m in good]
).reshape(-1,1,2)
dst=np.float32([kp2[m.trainIdx].pt for m in good]
).reshape(-1,1,2)
H,mask=cv2.findHomography(src,dst,cv2.RANSAC,5.0)
inliers=mask.ravel().sum()
print("Inliers:",inliers)
```

11. IMAGE FUSION / CT/MRI**# SAME SIZE FIRST**

```
mri=cv2.resize(mri,(ct.shape[1],ct.shape[0]))
```

50/50 fusion

```
fused=cv2.addWeighted(ct,0.5,mri,0.5,0)
```

70/30

```
fused=cv2.addWeighted(ct,0.7,mri,0.3,0)
```

MAX-INTENSITY FUSION

```
img2=cv2.resize(img2,(img1.shape[1],img1.shape[0]))
fused=np.maximum(img1,img2)
```

useful when keeping strongest intensity**# WEIGHT RULE**

```
# output = alpha*A + beta*B + gamma
```

```
# alpha+beta often = 1
```

```
fused=cv2.addWeighted(A,alpha,B,beta,0)
```

12. VIDEO / FRAME PROCESSING / MOTION**# READ VIDEO**

```
cap=cv2.VideoCapture("video.mp4")
while True:
ret,frame=cap.read()
```

```
if not ret: break
gray=cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
edges=cv2.Canny(gray,50,150)
cv2.imshow("Edges",edges)
if cv2.waitKey(1)&0xFF==ord("q"): break
cap.release(); cv2.destroyAllWindows()
```

MOTION BY FRAME DIFFERENCE

```
previous=None
```

inside loop:

```
gray=cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
if previous is not None:
diff=cv2.absdiff(previous,gray)
_,th=cv2.threshold(diff,30,255,cv2.THRESH_BINARY)
previous=gray
```

SAVE VIDEO

```
fourcc=cv2.VideoWriter_fourcc(*'mp4v')
out=cv2.VideoWriter('out.mp4',fourcc,20,
(frame.shape[1],frame.shape[0]))
# inside loop: out.write(frame)
out.release()
```

13. WATERSHED - TOUCHING OBJECTS

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
_,binary=cv2.threshold(gray,0,255,
cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel=np.ones((3,3),np.uint8)
opening=cv2.morphologyEx(binary,cv2.MORPH_OPEN,kernel,
iterations=2)
sure_bg=cv2.dilate(opening,kernel,iterations=3)

dist=cv2.distanceTransform(opening,cv2.DIST_L2,5)
_,sure_fg=cv2.threshold(dist,0.5*dist.max(),255,0)
sure_fg=np.uint8(sure_fg)
unknown=cv2.subtract(sure_bg,sure_fg)
n,markers=cv2.connectedComponents(sure_fg)
markers=markers+1
```

```

markers[unknown==255]=0

markers=cv2.watershed(img,markers)
img[markers==-1]=(255,0,0)
# idea:
# binary -> opening -> background
# -> distance transform -> foreground
# -> unknown -> connected components
# -> watershed

```

14. WAVELET / ANOMALY DETECTION

```

import pywt
coeffs=pywt.dwt2(gray, 'haar')
LL, (LH, HL, HH)=coeffs
# LL = low frequency
# LH/HL/HH = detail/high-frequency bands

detail=np.abs(HH)
threshold=detail.mean()+2*detail.std()
anomaly=(detail>threshold).astype(np.uint8)*255
# adjust 2 -> 3 for stricter detection

# RECONSTRUCT IF NEEDED
reconstructed=pywt.idwt2((LL, (LH, HL, HH)), 'haar')
reconstructed=np.uint8(np.clip(reconstructed, 0, 255))

```

15. SECURITY ZONE / POINT INSIDE POLYGON

```

# POINT-IN-ZONE
zone=np.array([[100,100],[500,100],
[500,400],[100,400]],np.int32)
point=(x,y)
inside=cv2.pointPolygonTest(
zone,point,False)
if inside>=0:
print("INSIDE")
else:
print("OUTSIDE")

```

DRAW_ZONE

```

cv2.polylines(img,[zone],True,(0,255,0),2)
cv2.circle(img,point,6,(0,0,255),-1)
# test many points:
for p in points:
if cv2.pointPolygonTest(zone,tuple(p),False)>=0:
print("inside:",p)

```

COMBINE WITH MOTION

```

# motion mask + pointPolygonTest
# = trigger only when motion is inside
# security zone

```

16. X-RAY / MEDICAL IMAGE MODEL PIPELINE

ENHANCEMENT TEMPLATE

```

gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
equalized=cv2.equalizeHist(gray)
g=gray.astype(np.float32)
c=255/np.log(1+np.max(g))
log_img=np.uint8(c*np.log(1+g))
adaptive=cv2.adaptiveThreshold(
equalized,255,cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY,11,2)

```

ROI + ENHANCEMENT

```

mask=np.zeros(gray.shape,np.uint8)
pts=np.array([[100,100],[500,100],
[500,400],[100,400]],np.int32)
cv2.fillPoly(mask,[pts],255)
roi=cv2.bitwise_and(equalized,equalized,mask=mask)

```

ALWAYS KEEP ORIGINAL IF NEEDED

```

gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
enhanced = cv2.equalizeHist(gray)
# use gray for another independent transform
# use enhanced for thresholding/segmentation

```

17. HIGH-VALUE 'TWIST' TEMPLATES**# FIND ONLY LARGE OBJECTS**

```
_,b=cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
cs,_=cv2.findContours(b,cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
large=[c for c in cs if cv2.contourArea(c)>1000]
```

CROP DETECTED BOX

```
x,y,w,h=cv2.boundingRect(c)
object_img=img[y:y+h,x:x+w]
```

filter by aspect ratio

```
aspect=w/h
if 0.5<aspect<2.0: print("candidate")
```

EDGE INSIDE ROI

```
roi=cv2.bitwise_and(gray,gray,mask=mask)
blur=cv2.GaussianBlur(roi,(5,5),0)
edges=cv2.Canny(blur,50,150)
```

common twist: mask FIRST, then process**18. PARAMETERS TO CHANGE IN EXAM****threshold:**

```
127 -> fixed global
0 + OTSU -> automatic
11,2 -> adaptive block/C
```

Canny:

```
50,150 -> lower/upper edge thresholds
```

Gaussian:

```
(5,5) -> kernel; must be odd
```

HoughLinesP:

```
threshold=50
minLineLength=50
maxLineGap=20
HoughCircles:
dp=1, minDist=30
param1=100, param2=30
minRadius/maxRadius
```

SIFT ratio:

```
0.75 -> stricter if lower
```

RANSAC:

```
5.0 -> reprojection threshold
```

K-means:

```
K=2/3/4 -> number of clusters
```

Watershed:

```
0.5*dist.max() -> foreground cutoff
```

19. ONE-LINE EXAM MEMORY

```
3 corresponding points = AFFINE
```

```
4+ corresponding points = HOMOGRAPHY
```

```
H = warpPerspective
```

```
Affine M = warpAffine
```

```
Edges = CANNY
```

```
Lines = HoughLinesP
```

```
Circles = HoughCircles
```

```
Feature matching = SIFT + ratio test
```

```
Uneven illumination = ADAPTIVE
```

```
Automatic threshold = OTSU
```

```
Small noise = OPENING
```

```
Small holes/gaps = CLOSING
```

```
Touching objects = WATERSHED
```

```
Bad matches/outliers = RANSAC
```

```
ROI application = bitwise_and(..., mask=mask)
```

PANORAMA =

```
SIFT -> match -> ratio test -> Homography
```

```
-> warpPerspective -> stitch
```

LANE =

```
gray -> blur -> Canny -> ROI -> HoughLinesP
```

OBJECT =

```
threshold -> contours -> area/box/centroid
```